

SVX Library Class Catalog

SystemVerilog Extension Library — Applications (**apps/**)

Version 2.0.1 beta

Mark Glasser

September 2026

Contents

1	Clock Generator	2
1.1	clk_descriptor #(uint32_t N=1)	2
	clk_gen.sv	3
1.2	clk_behavior #(uint32_t N=1)	3
1.3	clk_processor #(uint32_t N=1)	3
2	Memory Model	5
	mem_types.svh	5
2.1	mem_base #(uint32_t ADDR_BITS=32, uint32_t PAGE_BITS=16, uint32_t BLOCK_BITS=8, uint32_t WORD_SIZE=4)	5
2.2	mem_block #(uint32_t ADDR_BITS=32, uint32_t PAGE_BITS=16, uint32_t BLOCK_BITS=8, uint32_t WORD_SIZE=4)	6
2.3	mem_page #(uint32_t ADDR_BITS=32, uint32_t PAGE_BITS=16, uint32_t BLOCK_BITS=8, uint32_t WORD_SIZE=4)	7
2.4	mem #(uint32_t ADDR_BITS=32, uint32_t PAGE_BITS=16, uint32_t BLOCK_BITS=8, uint32_t WORD_SIZE=4)	8
2.5	mem_bounded #(uint32_t ADDR_BITS=32, uint32_t PAGE_BITS=16, uint32_t BLOCK_BITS=8, uint32_t WORD_SIZE=4)	10
	mem.sv	11
2.6	restrict_traits	11
3	Memory Map	12
3.1	mem_space #(uint32_t ADDR_SIZE=32)	12
3.2	mem_field #(uint32_t ADDR_SIZE=32)	13
3.3	mem_register #(uint32_t ADDR_SIZE=32)	14
3.4	mem_memory #(uint32_t ADDR_SIZE=32)	14
3.5	mem_region #(uint32_t ADDR_SIZE=32)	14
3.6	mem_view #(uint32_t ADDR_SIZE=32)	15
4	Priority Queue	16
4.1	pri_queue #(type T=int, type P=void_traits)	16
5	RPN Calculator	17
5.1	calc	17

Clock Generator

1.1 `clk_descriptor #(uint32_t N=1)`

Kind: class

Extends: object

Holds all configuration data for a single clock signal, including name, frequency, duty cycle, high/low times, time scale, and virtual interface handle, and validates those parameters before clock startup.

Constructor: `new()`

Method	Returns	Arguments	Description
<code>set_vif</code>	<code>void</code>	<code>vif, index</code>	Stores the virtual clock interface and the clock index within the interface signal array.
<code>set_name</code>	<code>void</code>	<code>n</code>	Stores the clock's display name.
<code>set_start_event</code>	<code>void</code>	<code>e</code>	Stores the event used to trigger clock startup.
<code>set_duty_cycle</code>	<code>void</code>	<code>dc</code>	Sets the duty cycle, accepting only values strictly greater than 0.0 and at most 1.0.
<code>set_freq</code>	<code>void</code>	<code>f</code>	Sets the clock frequency in Hz.
<code>set_times</code>	<code>void</code>	<code>hi, lo</code>	Directly sets the clock high-time and low-time values.
<code>set_scale</code>	<code>void</code>	<code>s</code>	Sets the time-scale enum used when computing period from frequency.
<code>set_initial_delay</code>	<code>void</code>	<code>t</code>	Sets the one-time delay applied before the clock begins toggling.
<code>set_verbose</code>	<code>void</code>	—	Enables verbose diagnostic messages for this clock.
<code>clr_verbose</code>	<code>void</code>	—	Disables verbose diagnostic messages for this clock.
<code>scale_str</code>	<code>string</code>	—	Returns the abbreviation string corresponding to the current <code>time_scale</code> enum value.

(continued on next page...)

(continued...)

Method	Returns	Arguments	Description
<code>validate</code>	<code>bit</code>	—	Validates all descriptor fields, computing <code>time_hi</code> and <code>time_lo</code> from frequency and duty cycle when they have not been set directly, and returns 1 if the descriptor is fully consistent.
<code>to_str</code>	<code>string</code>	—	Returns a human-readable summary string of this clock descriptor.
<code>compare</code>	<code>int32_t</code>	<code>obj</code>	Compares this descriptor to another by <code>clk_index</code> , returning 0, 1, or -1 for ordering purposes.
<code>copy</code>	<code>object</code>	<code>rhs</code>	Copies all fields from the given <code>clk_descriptor</code> into this one and returns the result.

`clk_gen.sv`

Package wrapper that includes all clock generator source files.

1.2 `clk_behavior #(uint32_t N=1)`

Kind: class

Extends: `task_behavior#(clk_descriptor#(N))`

Implements the task body that drives a single clock pin by waiting for the start event and then toggling the bound virtual interface signal forever according to the descriptor's `time_hi` and `time_lo` values.

Constructor: (none)

Method	Returns	Arguments	Description
<code>tsk</code>	<code>void</code>	—	Drives the associated clock signal by alternating low and high levels on the virtual interface at the intervals specified in the bound <code>clk_descriptor</code> .

1.3 `clk_processor #(uint32_t N=1)`

Kind: class

Implements: `process_if`

Manages a collection of clock-driving processes by validating a vector of `clk_descriptors`, spawning one `clk_behavior` task per descriptor in a `process_group`, and delegating suspend/resume/kill/await control to that group.

Constructor: (none)

Method	Returns	Arguments	Description
<code>set_vector</code>	<code>void</code>	<code>v</code>	Validates each <code>clk_descriptor</code> in the supplied vector and, if all pass, stores the vector for use during <code>exec</code> .
<code>print_descriptors</code>	<code>void</code>	—	Iterates over the stored clock descriptor vector and prints each descriptor's <code>to_str</code> output.
<code>exec</code>	<code>void</code>	—	Kills any running clock processes, then creates and starts a new <code>process_group</code> containing one process per clock descriptor.
<code>suspend</code>	<code>void</code>	—	Delegates suspension of all clock processes to the underlying <code>process_group</code> .
<code>resume</code>	<code>void</code>	—	Delegates resumption of all clock processes to the underlying <code>process_group</code> .
<code>kill</code>	<code>void</code>	—	Kills all clock processes via the <code>process_group</code> and sets the group handle to null.
<code>is_done</code>	<code>bit</code>	—	Returns whether all clock processes in the group have completed.
<code>await</code>	<code>void</code>	—	Blocks until all clock processes in the group have completed.

Memory Model

`mem_types.svh`

Defines type aliases and enumerations used throughout the memory model.

2.1 `mem_base #(uint32_t ADDR_BITS=32, uint32_t PAGE_BITS=16, uint32_t BLOCK_BITS=8, uint32_t WORD_SIZE=4)`

Kind: virtual class

Extends: object

Parameterized abstract base class for all memory model components, supplying address-field decomposition helpers, a per-component security map with restriction get/set/clear, and pure-virtual interfaces for read, write, security interrogation, and dump.

Constructor: `new(root)`

Method	Returns	Arguments	Description
<code>get_page_key</code>	<code>page_key_t</code>	<code>addr</code>	Extracts and returns the page-index field from a full address.
<code>get_block_addr</code>	<code>block_addr_t</code>	<code>addr</code>	Extracts and returns the block-address field from a full address.
<code>get_byte_addr</code>	<code>byte_addr_t</code>	<code>addr</code>	Extracts and returns the byte-address field from a full address.
<code>get_aligned_byte_addr</code>	<code>byte_addr_t</code>	<code>addr</code>	Returns the word-aligned byte address by masking off the low-order word-offset bits.
<code>is_word_aligned</code>	<code>bit</code>	<code>addr</code>	Returns 1 if the address is aligned to a word boundary, 0 otherwise.
<code>construct_addr</code>	<code>addr_t</code>	<code>page_key</code> , <code>block_addr</code> , <code>byte_addr</code>	Assembles a full address from its constituent page, block, and byte fields.
<code>read</code>	<code>word_t</code>	<code>addr</code>	Pure virtual: reads one word from the given address.
<code>write</code>	<code>void</code>	<code>addr</code> , <code>data</code>	Pure virtual: writes one word to the given address.

(continued on next page...)

(continued...)

Method	Returns	Arguments	Description
<code>read_byte</code>	<code>byte_t</code>	<code>addr</code>	Pure virtual: reads one byte from the given address.
<code>write_byte</code>	<code>void</code>	<code>addr, data</code>	Pure virtual: writes one byte to the given address.
<code>set_error</code>	<code>void</code>	<code>err</code>	Forwards an error code to the root memory object's error-reporting interface.
<code>set_restriction</code>	<code>void</code>	<code>addr, r</code>	Inserts or updates an access restriction for the given address in this component's security map.
<code>get_restriction</code>	<code>restrict_t</code>	<code>addr</code>	Retrieves the access restriction associated with the given address from this component's security map.
<code>clear_restriction</code>	<code>void</code>	<code>addr</code>	Removes the access restriction entry for the given address from this component's security map.
<code>get_addr_restriction</code>	<code>restrict_t</code>	<code>addr</code>	Pure virtual: returns the effective security restriction for the given address, searching all levels of the memory hierarchy.
<code>dump_security</code>	<code>void</code>	<code>addr</code>	Pure virtual: prints this component's security map.
<code>dump</code>	<code>void</code>	<code>addr</code>	Pure virtual: prints this component's memory contents.
<code>show_addr</code>	<code>void</code>	<code>addr</code>	Displays all decomposed fields of the given address for debugging purposes.
<code>show</code>	<code>void</code>	—	Displays the current memory organization parameters and address-field masks.

2.2 `mem_block # (uint32_t ADDR_BITS=32, uint32_t PAGE_BITS=16, uint32_t BLOCK_BITS=8, uint32_t WORD_SIZE=4)`

Kind: class

Extends: `mem_base#(ADDR_BITS, PAGE_BITS, BLOCK_BITS, WORD_SIZE)`

Implements a memory block as a sparse byte vector, performing big-endian word-aligned and byte-level reads and writes with word-granularity security restriction enforcement.

Constructor: `new(root)`

Method	Returns	Arguments	Description
<code>write</code>	<code>void</code>	<code>addr, data</code>	Writes one word big-endian into the byte vector, enforcing word alignment and write restrictions.
<code>read</code>	<code>word_t</code>	<code>addr</code>	Reads one word big-endian from the byte vector, enforcing word alignment and read restrictions.
<code>read_byte</code>	<code>byte_t</code>	<code>addr</code>	Reads one byte from the byte vector, enforcing byte-level read restrictions.
<code>write_byte</code>	<code>void</code>	<code>addr, data</code>	Writes one byte into the byte vector, enforcing byte-level write restrictions.
<code>get_addr_restriction</code>	<code>restrict_t</code>	<code>addr</code>	Returns the word-level security restriction for the given address.
<code>dump_security</code>	<code>void</code>	<code>addr</code>	Prints all word-level security restrictions stored in this block's security map.
<code>dump</code>	<code>void</code>	<code>addr</code>	Prints all bytes stored in this block in hexadecimal, 16 bytes per line.

2.3 `mem_page # (uint32_t ADDR_BITS=32, uint32_t PAGE_BITS=16, uint32_t BLOCK_BITS=8, uint32_t WORD_SIZE=4)`

Kind: class

Extends: `mem_base#(ADDR_BITS, PAGE_BITS, BLOCK_BITS, WORD_SIZE)`

Implements a memory page as a sparse map of blocks, creating blocks on demand and enforcing block-granularity security restrictions before delegating reads and writes to the appropriate block.

Constructor: `new(root)`

Method	Returns	Arguments	Description
<code>write</code>	<code>void</code>	<code>addr, data</code>	Enforces block-level write restrictions and delegates a word write to the appropriate block, creating the block if it does not yet exist.
<code>read</code>	<code>word_t</code>	<code>addr</code>	Enforces block-level read restrictions and delegates a word read to the appropriate block, returning 0 if the block is absent.
<code>write_byte</code>	<code>void</code>	<code>addr, data</code>	Enforces block-level write restrictions and delegates a byte write to the appropriate block, creating the block if necessary.

(continued on next page...)

(continued...)

Method	Returns	Arguments	Description
<code>read_byte</code>	<code>byte_t</code>	<code>addr</code>	Enforces block-level read restrictions and delegates a byte read to the appropriate block, returning 0 if the block is absent.
<code>set_word_restriction</code>	<code>void</code>	<code>addr, r</code>	Sets a word-level security restriction in the block identified by the given address, creating the block if necessary.
<code>clear_word_restriction</code>	<code>void</code>	<code>addr</code>	Clears a word-level security restriction in the block identified by the given address.
<code>get_addr_restriction</code>	<code>restrict_t</code>	<code>addr</code>	Returns the effective restriction for the given address, checking block-level then word-level security maps.
<code>dump_security</code>	<code>void</code>	<code>addr</code>	Prints this page's block-level security map and then recursively dumps each block's word-level security map.
<code>dump</code>	<code>void</code>	<code>addr</code>	Iterates over all blocks in the page and dumps each one's byte contents.

2.4 `mem #(uint32_t ADDR_BITS=32, uint32_t PAGE_BITS=16, uint32_t BLOCK_BITS=8, uint32_t WORD_SIZE=4)`

Kind: class

Extends: `mem_base#(ADDR_BITS, PAGE_BITS, BLOCK_BITS, WORD_SIZE)`

Top-level sparse memory model that organizes pages in a map, enforces three-level (page, block, word) security restrictions on every access, and provides a comprehensive error-reporting interface.

Constructor: `new()`

Method	Returns	Arguments	Description
<code>write</code>	<code>void</code>	<code>addr, data</code>	Checks page-level security, then writes a word to the addressed page, creating the page if absent.
<code>read</code>	<code>word_t</code>	<code>addr</code>	Checks page-level security, then reads a word from the addressed page, returning 0 if the page is absent.

(continued on next page...)

(continued...)

Method	Returns	Arguments	Description
<code>write_byte</code>	<code>void</code>	<code>addr, data</code>	Checks page-level security, then writes a byte to the addressed page, creating the page if absent.
<code>read_byte</code>	<code>byte_t</code>	<code>addr</code>	Checks page-level security, then reads a byte from the addressed page, returning 0 if the page is absent.
<code>set_page_restriction</code>	<code>void</code>	<code>addr, r</code>	Sets a page-level security restriction for the page that contains the given address.
<code>get_page_restriction</code>	<code>restrict_t</code>	<code>addr</code>	Returns the page-level security restriction for the page that contains the given address.
<code>clear_page_restriction</code>	<code>void</code>	<code>addr</code>	Clears the page-level security restriction for the page that contains the given address.
<code>set_block_restriction</code>	<code>void</code>	<code>addr, r</code>	Sets a block-level security restriction in the page identified by the given address, creating the page if absent.
<code>clear_block_restriction</code>	<code>void</code>	<code>addr</code>	Clears a block-level security restriction in the page identified by the given address.
<code>set_word_restriction</code>	<code>void</code>	<code>addr, r</code>	Sets a word-level security restriction in the appropriate page and block, creating them if absent.
<code>clear_word_restriction</code>	<code>void</code>	<code>addr</code>	Clears a word-level security restriction in the page and block identified by the given address.
<code>is_readable</code>	<code>bit</code>	<code>addr</code>	Returns 1 if the given address has no read restriction active at any level.
<code>is_writable</code>	<code>bit</code>	<code>addr</code>	Returns 1 if the given address has no write restriction active at any level.
<code>get_addr_restriction</code>	<code>restrict_t</code>	<code>addr</code>	Returns the most restrictive security restriction for the given address across all three hierarchy levels.
<code>set_last_error</code>	<code>void</code>	<code>err</code>	Records the most recent error code; called internally by <code>mem_base::set_error</code> .
<code>last_operation_failed</code>	<code>bit</code>	—	Returns 1 if the most recent memory operation resulted in any error.

(continued on next page...)

(continued...)

Method	Returns	Arguments	Description
<code>get_last_error</code>	<code>error_t</code>	—	Returns the <code>error_t</code> enum value from the most recent operation.
<code>get_last_op_string</code>	<code>string</code>	—	Returns a formatted string describing the last operation, its address, and the associated error condition.
<code>dump_security</code>	<code>void</code>	<code>addr</code>	Prints the complete security map for all pages and their blocks.
<code>dump</code>	<code>void</code>	<code>addr</code>	Traverses all pages in order and dumps each page's byte contents.

2.5 `mem_bounded` `#(uint32_t ADDR_BITS=32, uint32_t PAGE_BITS=16, uint32_t BLOCK_BITS=8, uint32_t WORD_SIZE=4)`

Kind: class

Extends: `mem#(ADDR_BITS, PAGE_BITS, BLOCK_BITS, WORD_SIZE)`

Extends the top-level sparse memory with configurable lower and upper address bounds, rejecting any read or write access that falls outside the defined range and locking the bounds on first access.

Constructor: `new(lower, upper)`

Method	Returns	Arguments	Description
<code>set_bounds</code>	<code>void</code>	<code>lower, upper</code>	Sets both the lower and upper address bounds, provided the bounds have not yet been locked.
<code>set_lower_bound</code>	<code>void</code>	<code>lower</code>	Sets the lower address bound if the bounds are not yet locked.
<code>set_upper_bound</code>	<code>void</code>	<code>upper</code>	Sets the upper address bound if the bounds are not yet locked.
<code>get_lower_bound</code>	<code>addr_t</code>	—	Returns the current lower address bound.
<code>get_upper_bound</code>	<code>addr_t</code>	—	Returns the current upper address bound.
<code>set_bounds_lock</code>	<code>void</code>	—	Locks the bounds, preventing further modification.
<code>get_bounds_lock</code>	<code>bit</code>	—	Returns 1 if the bounds are locked, 0 otherwise.
<code>write</code>	<code>void</code>	<code>addr, data</code>	Locks bounds, validates the address is within range, then delegates the word write to the parent mem class.

(continued on next page...)

(continued...)

Method	Returns	Arguments	Description
<code>read</code>	<code>word_t</code>	<code>addr</code>	Locks bounds, validates the address is within range, then delegates the word read to the parent mem class.
<code>write_byte</code>	<code>void</code>	<code>addr, data</code>	Locks bounds, validates the address is within range, then delegates the byte write to the parent mem class.
<code>read_byte</code>	<code>byte_t</code>	<code>addr</code>	Locks bounds, validates the address is within range, then delegates the byte read to the parent mem class.
<code>dump</code>	<code>void</code>	<code>addr</code>	Prints the lower and upper bounds, then delegates to the parent dump.

`mem.sv`

Package wrapper that includes all memory model source files.

2.6 `restrict_traits`

Kind: class

Extends: `void_t`

Traits adapter for the `restrict_t` enum type that supplies the equality, comparison, and sort operations required by SVX parameterized container classes.

Constructor: (none)

Method	Returns	Arguments	Description
<code>equal</code>	<code>bit</code>	<code>a, b</code>	Static: returns 1 if the two <code>restrict_t</code> values are equal.
<code>compare</code>	<code>int32_t</code>	<code>a, b</code>	Static: returns 1, -1, or 0 to indicate the ordering relationship between two <code>restrict_t</code> values.
<code>sort</code>	<code>void</code>	<code>vec</code>	Static: sorts a queue of <code>restrict_t</code> values in ascending order.

Memory Map

3.1 mem_space #(uint32_t ADDR_SIZE=32)

Kind: virtual class

Extends: tree

Abstract base class for all nodes in a hierarchical address map, providing offset/size storage, absolute-address calculation, sibling overlap detection, recursive address search, and a hierarchical dump.

Constructor: new(name, parent, _type, _offset, _size)

Method	Returns	Arguments	Description
insert_space	void	space	Adds a child memory space to this node after verifying the child type is permitted.
get_type	mem_space_type_t	—	Pure virtual: returns the enum identifying the kind of memory space this object represents.
check_child	bit	child	Pure virtual: returns 1 if the given child type is permitted under this memory space type.
get_offset	addr_t	—	Returns the byte offset of this space from its parent.
get_size	mem_size_t	—	Returns the size of this memory space in bytes (or bits for a field).
get_addr	addr_t	—	Returns the calculated absolute address of this memory space.
get_end_addr	addr_t	—	Returns the last address occupied by this memory space.
get_type_name	string	—	Returns a string name for the mem_space_type enum of this object.
get_error	bit	—	Returns 1 if this memory space has been flagged as erroneous.
clear_error	void	—	Clears the error flag on this memory space.

(continued on next page...)

(continued...)

Method	Returns	Arguments	Description
<code>to_str</code>	<code>string</code>	—	Returns a formatted string showing the type, absolute address range, offset, size, and hierarchical name of this space.
<code>calculate_and_check</code>	<code>bit</code>	—	Computes absolute addresses for the entire subtree and then performs consistency checks, returning 1 if all checks pass.
<code>calculate</code>	<code>void</code>	<code>base_addr</code>	Recursively calculates and stores the absolute address for this space and all its descendants.
<code>check</code>	<code>bit</code>	—	Recursively checks this space and all descendants for hierarchy violations and sibling overlaps, returning 1 if no errors are found.
<code>check_overlap</code>	<code>bit</code>	—	Checks all pairs of sibling spaces under this node for illegal address overlaps, flagging and reporting any that are found.
<code>overlaps</code>	<code>bit</code>	<code>space, sibling</code>	Returns 1 if the two given memory spaces have overlapping address ranges.
<code>find_addr</code>	<code>list_t</code>	<code>search_addr</code>	Returns a list of all leaf memory spaces (registers and memories) whose address range contains the given address.
<code>find_addr_all</code>	<code>list_t</code>	<code>search_addr</code>	Returns a list of all memory spaces at any hierarchy level whose address range contains the given address.
<code>find_addr_recurse</code>	<code>void</code>	<code>search_addr,</code> <code>list, all</code>	Recursively traverses the hierarchy to collect memory spaces containing the given address into the provided list.
<code>dump</code>	<code>void</code>	—	Prints a flat listing of all memory spaces in the subtree using a tree iterator.

3.2 `mem_field # (uint32_t ADDR_SIZE=32)`

Kind: class

Extends: `mem_space#(ADDR_SIZE)`

Represents a bit field within a register, recording a bit offset and bit width, and prohibiting any children.

Constructor: `new(name, parent, _offset, _size)`

Method	Returns	Arguments	Description
<code>get_type</code>	<code>mem_space_type_t</code>	—	Returns the FIELD enum constant.
<code>check_child</code>	<code>bit</code>	<code>child</code>	Always returns 0 because fields cannot contain children.

3.3 `mem_register #(uint32_t ADDR_SIZE=32)`

Kind: class

Extends: `mem_space#(ADDR_SIZE)`

Represents a hardware register in the address map, allowing only `mem_field` children and providing a convenience method to add fields by name.

Constructor: `new(name, parent, _offset, _size)`

Method	Returns	Arguments	Description
<code>get_type</code>	<code>mem_space_type_t</code>	—	Returns the REGISTER enum constant.
<code>check_child</code>	<code>bit</code>	<code>child</code>	Returns 1 only if the child is a FIELD.
<code>add_field</code>	<code>void</code>	<code>name</code> , <code>_offset</code> , <code>_size</code>	Creates and attaches a new <code>mem_field</code> child with the given name, bit offset, and bit size.

3.4 `mem_memory #(uint32_t ADDR_SIZE=32)`

Kind: class

Extends: `mem_space#(ADDR_SIZE)`

Represents a contiguous memory region (RAM or ROM) in the address map as a leaf node that cannot contain children.

Constructor: `new(name, parent, _offset, _size)`

Method	Returns	Arguments	Description
<code>get_type</code>	<code>mem_space_type_t</code>	—	Returns the MEMORY enum constant.
<code>check_child</code>	<code>bit</code>	<code>child</code>	Always returns 0 because memory spaces cannot contain children.

3.5 `mem_region #(uint32_t ADDR_SIZE=32)`

Kind: class

Extends: `mem_space#(ADDR_SIZE)`

Represents a named address region that may contain registers, memories, sub-regions, and views but not fields, with convenience methods to add each child type.

Constructor: `new(name, parent, _offset, _size)`

Method	Returns	Arguments	Description
<code>get_type</code>	<code>mem_space_type_t</code>	—	Returns the REGION enum constant.
<code>check_child</code>	<code>bit</code>	<code>child</code>	Returns 1 for any child type except FIELD.
<code>add_register</code>	<code>void</code>	<code>name</code> , <code>_offset</code> , <code>_size</code>	Creates and attaches a new <code>mem_register</code> child.
<code>add_memory</code>	<code>void</code>	<code>name</code> , <code>_offset</code> , <code>_size</code>	Creates and attaches a new <code>mem_memory</code> child.
<code>add_region</code>	<code>void</code>	<code>name</code> , <code>_offset</code> , <code>_size</code>	Creates and attaches a new <code>mem_region</code> child.
<code>add_view</code>	<code>void</code>	<code>name</code> , <code>_offset</code> , <code>_size</code>	Creates and attaches a new <code>mem_view</code> child.

3.6 `mem_view #(uint32_t ADDR_SIZE=32)`

Kind: class

Extends: `mem_space#(ADDR_SIZE)`

Represents an alternate view of a memory region that shares the parent's base address and is permitted to overlap sibling spaces, with convenience methods to add child registers, memories, regions, and views.

Constructor: `new(name, parent, _offset, _size)`

Method	Returns	Arguments	Description
<code>get_type</code>	<code>mem_space_type_t</code>	—	Returns the VIEW enum constant.
<code>check_child</code>	<code>bit</code>	<code>child</code>	Returns 1 for any child type except FIELD.
<code>add_register</code>	<code>void</code>	<code>name</code> , <code>_offset</code> , <code>_size</code>	Creates and attaches a new <code>mem_register</code> child.
<code>add_memory</code>	<code>void</code>	<code>name</code> , <code>_offset</code> , <code>_size</code>	Creates and attaches a new <code>mem_memory</code> child.
<code>add_region</code>	<code>void</code>	<code>name</code> , <code>_offset</code> , <code>_size</code>	Creates and attaches a new <code>mem_region</code> child.
<code>add_view</code>	<code>void</code>	<code>name</code> , <code>_offset</code> , <code>_size</code>	Creates and attaches a new <code>mem_view</code> child.

Priority Queue

4.1 `pri_queue #(type T=int, type P=void_traits)`

Kind: class

Generic priority queue implemented as a map of FIFO queues keyed by priority value, returning the highest-priority item on each pop while preserving insertion order among equal-priority items.

Constructor: `new()`

Method	Returns	Arguments	Description
<code>is_empty</code>	<code>bit</code>	—	Returns 1 if the priority queue contains no items.
<code>push</code>	<code>void</code>	<code>pri, item</code>	Inserts an item at the given priority level, creating a new queue for that priority if one does not already exist.
<code>pop</code>	<code>T</code>	—	Removes and returns the item with the highest priority, deleting the priority's queue when it becomes empty.

RPN Calculator

5.1 calc

Kind: class

Reverse Polish Notation calculator that lexes an input string into tokens and evaluates the expression using a stack, supporting integer and floating-point addition, subtraction, multiplication, and division.

Constructor: `new()`

Method	Returns	Arguments	Description
<code>get_last_result</code>	<code>item_t</code>	—	Returns the triple holding the token type and value produced by the most recent successful calculate call.
<code>print_item</code>	<code>void</code>	<code>t</code>	Prints the value of a result item, formatting it as an integer or floating-point number based on its token type.
<code>calculate</code>	<code>bit</code>	<code>s</code>	Parses and evaluates the given RPN expression string, printing the result and returning 1 on success or 0 on any error.